

```

# =====
# Student Name : Srushti Kale
# PRN          : 23070521067
# Experiment 5 : Hyperparameter Tuning and Model Optimization
# =====

import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# --- 1. Load and Preprocess Dataset ---
# Srushti Kale - 23070521067: Using Breast Cancer dataset for classification
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# --- 2. Define a Model Building Function ---
# Srushti Kale - 23070521067: Function to easily swap hyperparameter values
def build_and_compile_model(num_layers, activation_fn, learning_rate, optimizer):
    model = keras.Sequential()
    model.add(keras.layers.InputLayer(input_shape=(X_train_scaled.shape[1],)))

    # Tuning Number of Layers and Activation Functions
    for _ in range(num_layers):
        model.add(keras.layers.Dense(32, activation=activation_fn))

    model.add(keras.layers.Dense(1, activation='sigmoid'))

    # Tuning Optimizers and Learning Rate
    if optimizer_name == 'adam':
        opt = keras.optimizers.Adam(learning_rate=learning_rate)
    elif optimizer_name == 'sgd':
        opt = keras.optimizers.SGD(learning_rate=learning_rate)

    model.compile(optimizer=opt, loss='binary_crossentropy', metrics=['accuracy'])
    return model

# --- 3. Define Configurations to Experiment With ---
# Tuning: Learning Rate, Batch Size, Layers, Activation, Optimizer
configurations = [
    {"name": "Config A (Baseline)", "layers": 1, "act": "relu", "lr": 0.001, "batch": 32},
    {"name": "Config B (Deeper, Tanh)", "layers": 3, "act": "tanh", "lr": 0.001, "batch": 32},
    {"name": "Config C (SGD, High LR, Batch 16)", "layers": 2, "act": "relu", "lr": 0.01, "batch": 16},
]

```

```

# --- 4. Train Models and Track History ---
history_dict = {}
EPOCHS = 40

print("Starting Hyperparameter Experiments...")
for config in configurations:
    print(f"Training {config['name']}...")
    model = build_and_compile_model(
        num_layers=config['layers'],
        activation_fn=config['act'],
        learning_rate=config['lr'],
        optimizer_name=config['opt']
    )

    # Tuning Batch Size during fit
    history = model.fit(
        X_train_scaled, y_train,
        epochs=EPOCHS,
        batch_size=config['batch'],
        validation_split=0.2,
        verbose=0 # Set to 0 to keep output clean, change to 1 to see epochs
    )
    history_dict[config['name']] = history

# --- 5. Compare Validation Accuracy ---
# Srushti Kale - 23070521067: Visualizing the impact of different hyperparameters
plt.figure(figsize=(10, 6))

colors = ['blue', 'green', 'red']
for (name, history), color in zip(history_dict.items(), colors):
    plt.plot(history.history['val_accuracy'], label=f"{name}", color=color, linestyle='solid')

plt.title('Model Optimization: Validation Accuracy Comparison\n(Srushti Kale)')
plt.xlabel('Epochs')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.grid(True)
plt.show()

```

```
Starting Hyperparameter Experiments...
Training Config A (Baseline)...
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/input_layer.py:
warnings.warn(
Training Config B (Deeper, Tanh)...
Training Config C (SGD, High LR, Batch 16)...
```

